Computer Architecture and System Software

Lecture 04: Intro to Assembly Language

Instructor:
David Bosc
Applied Computer Science
University of Winnipeg

Announcements

- Assignment 1 due this Friday, Jan 30th
- Going forward, please submit calculation work **exclusively** as **.PDF**.
  - Lab 4 will be floating point calculations, you need to put this in a word processor and export as PDF (pictures of work by hand ok, but needs to be attached in a PDF in the proper order)

Assembly Language Background

Assembly Language Programming

- We will be working with the Intel 32-bit architecture
- For work at home you will need an emulator, DOSBOX
  - http://www.dosbox.com/
- Other software packages available, but solutions given in class will only be guaranteed to compile using DOSBOX

High-Level Language

- High-level language (HLL) shield us from machine-level complexity
- HLL programs are
  - Easier to write
  - Less error prone
  - Readable
  - Portable
  - Efficient

High-Level Languages

- HLL relieves programmer from
  - Writing in machine-level instructions
  - Managing memory
  - Rewriting programs to run on different machines

Compiler

- Main work is done by compiler
  - Translate HLL program to assembly code (hardest part)
  - Assemble assembly code to machine code (object code)
  - Link object code with runtime library
  - Yields an executable
- Assembly code:
  - VERY close to machine code
  - Human readable encoding of machine code
  - Each instruction corresponds to a machine instruction
  - Not readable by machine (needs to be assembled)

Machine-Level (ML) Code

- Both assembly and machine code are ML code
- How does ML differ from High Level (HL) ?
- HL view:
  - Objects of different types are declared and allocated memory
  - Memory is accessed by accessing the objects
  - Local variables used to store temporary values
  - CPU state is hidden from programmer
  - Object type defines what operations can be performed
  - Single statement can perform a complex task
    - e.g. if (a+b < c) someFunc(d);

Machine-Level (ML) Code

- ML view:
  - CPU state is visible and accessible to the program
  - All CPU state is stored in its register file, *i.e.* the memory on the CPU
  - CPU has:
    - Registers
    - ALU
    - Control Unit
    - Execution Unit
    - *etc*

Motivation for Assembly Code

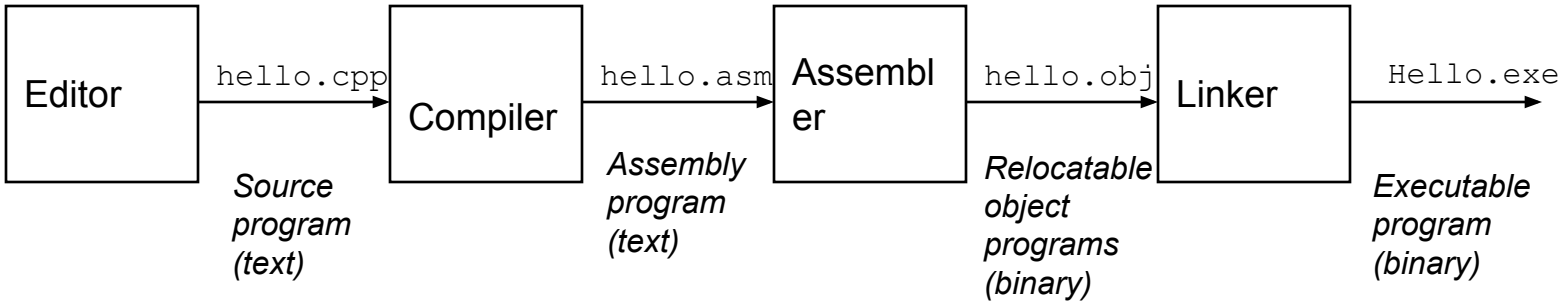
- Important:
  - Understand how computers execute code
  - Understand what optimizations the compiler performs
  - Reverse engineer a piece of software
  - Improve efficiency of a program
  - Write programs that have direct control over system hardware
    - Example: writing a device driver to a new scanner
    - If you are developing system software, you will need to know assembly language

Instruction Set

- The set of instructions (and their encodings) that a processor can execute
- We will use the IA32 (aka i386/i486/Pentium) instruction set
  - Used by majority of computers
    - Instruction set for Intel's most commercially successful microprocessors
  - Easily accessible
  - Large # of instructions and modes
  - Gives a good view of how computers work

Program Encoding

- Recall the compilation process



- hello.asm: assembly (human readable, not executable)
- hello.obj: machine code (binary, executable)
- Assembly and machine code are different encodings of the same instructions

x86 and IA-32 Instruction Set Architectures

History

- Intel first introduced microprocessors in 1969
- Work on early processors led to the development of the Intel Architecture (IA)
- First processor in the IA family was the 8086 (1979)
  - 20-bit address bus and 16-bit data bus
- First 32-bit IA processor was the 80386 (1985)
  - 32-bit address bus and 32-bit data bus
- Pentium introduced in 1993
  - Not named 80586 because numbers can't be trademarked
  - 32-bit address bus and 64-bit data bus

Source (UofW electronic resource): Chapter 3 in S. Dandamudi, *Introduction to Assembly Language Programming for Pentium and RISC Processors*, Springer, 2005.

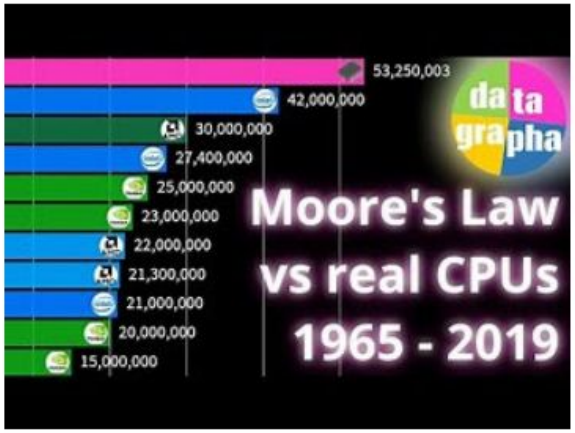
History

Processor	Year	Frequency (MHz)	Transistor count	Register width	Data bus width	Maximum address space
8086	1978	8	29 K	16	16	1 MB
80286	1982	12.5	134 K	16	16	16 MB
80386	1985	20	275 K	32	32	4 GB
80486	1989	25	1.2 M	32	32	4 GB
Pentium	1993	60	3.1 M	32	64	4 GB
Pentium Pro	1995	200	5.5 M	32	64	64 GB
Pentium II	1997	266	7 M	32	64	64 GB
Pentium III	1999	500	8.2 M	32	64	64 GB
Pentium 4	2000	1500	42 M	32	64	64 GB

Source (UofW electronic resource): Chapter 3 in S. Dandamudi, Introduction to Assembly Language Programming for Pentium and RISC Processors, Springer, 2005.

History

- https://www.youtube.com/watch?v=7uvUiq_jTLM



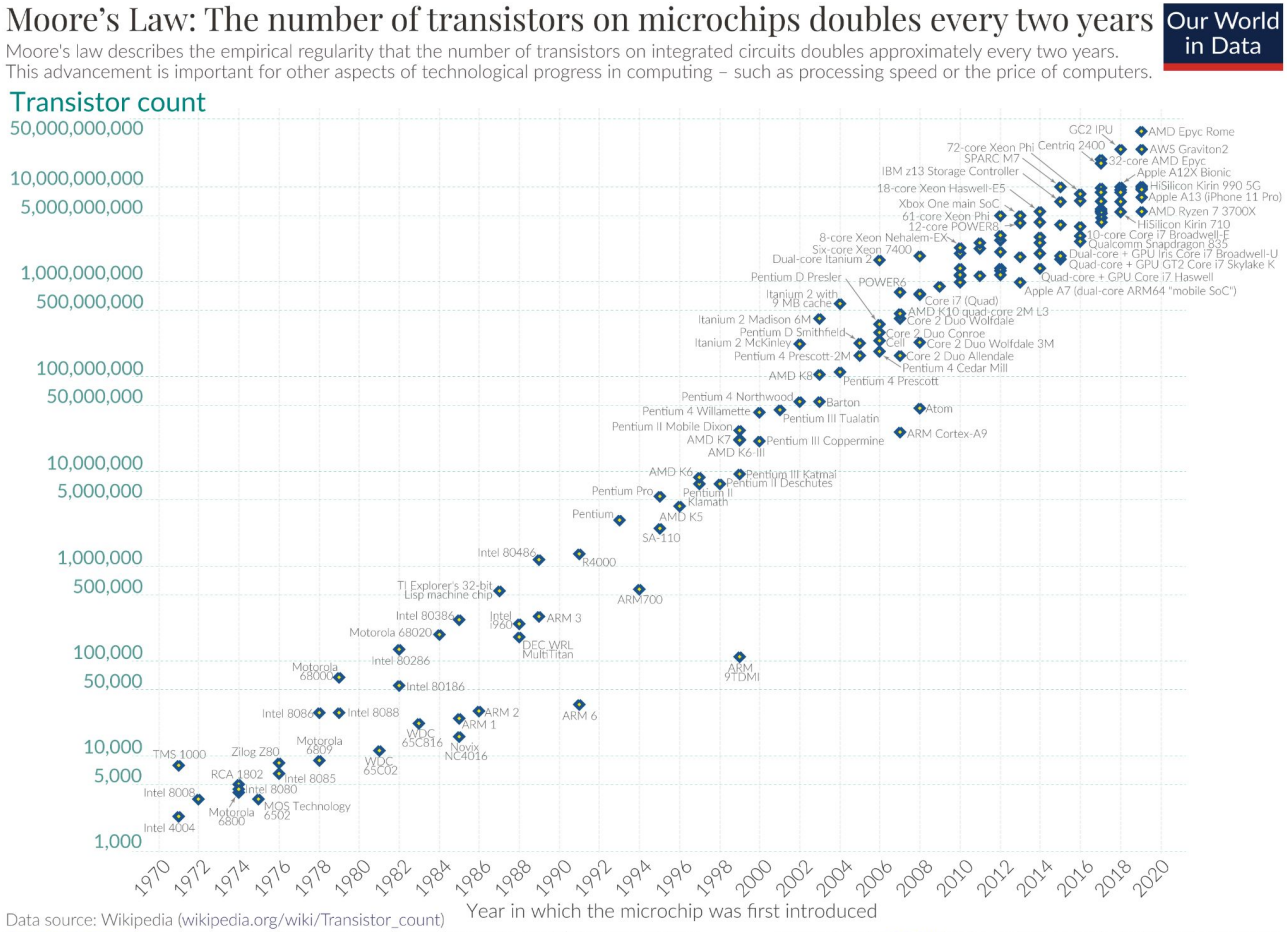
Moore's Law graphed vs real CPUs & GPUs 1965 - 2019

Moore's Law has been ridiculously accurate for more than 50 years - how long will it hold up? See in this visualization how the actual transistors in CPUs and GPUs compare to the linear progression of Moore's Law. From the early days of microprocessors, to Intel dominance and the rise of GPUs. Music: Dizaro - Aurora Borealis https ...

www.youtube.com

Moore's Law

https://en.wikipedia.org/wiki/Moore%27s_law

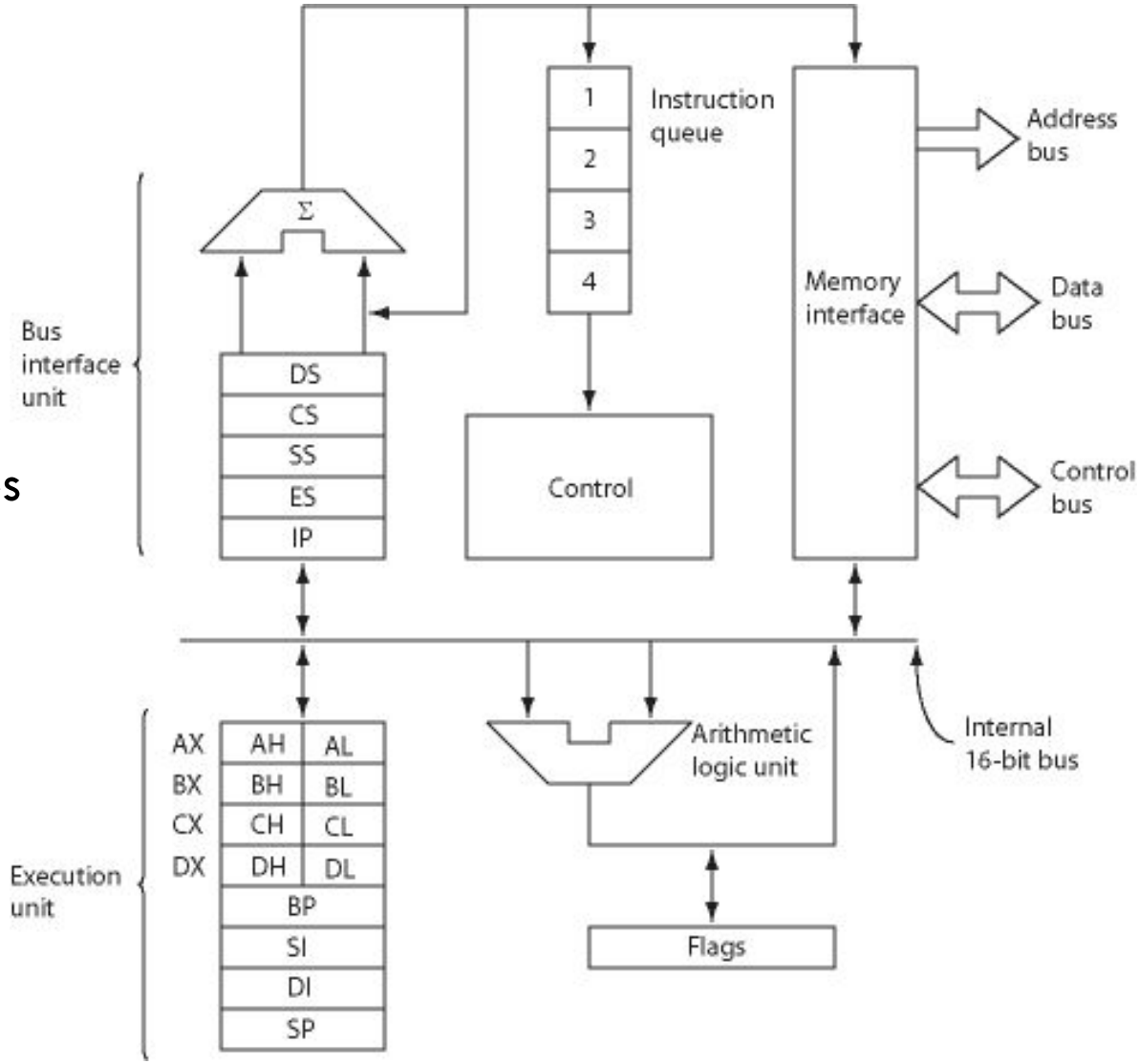


8086 / 8088 Architecture

- 8086/8088 is the first CPU in the X86-series (1979)
  - 8088 is identical to 8086 except it has a 8-bit data bus instead of a 16-bit data bus (more economical)
- Later processors in this series have similar but extended architectures
- Need to start at the beginning to understand a processor like Pentium-IV
  - Or newer ones like today's I5, I7 or I9

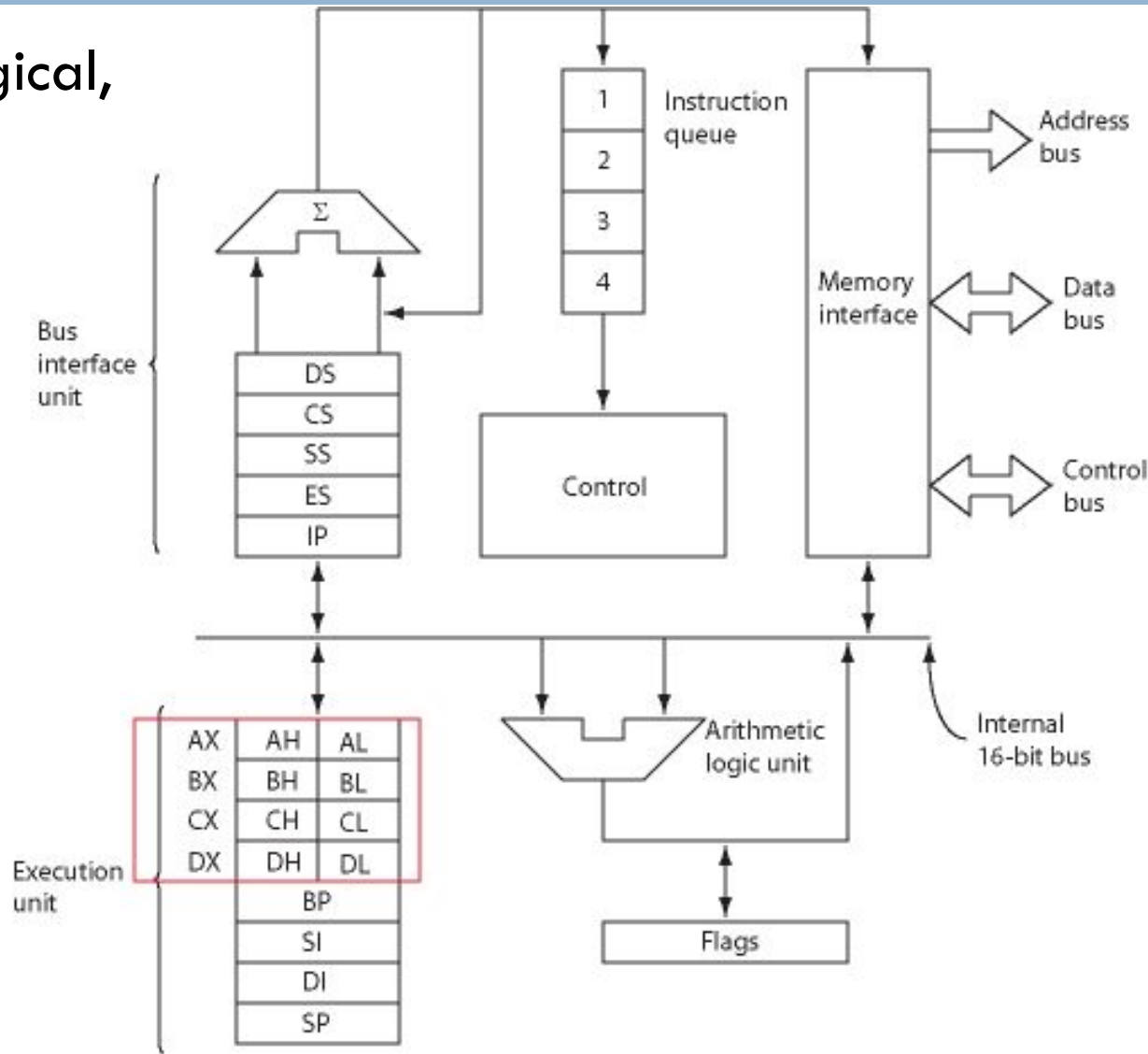
8086 / 8088 Architecture

- Two main units
  - Bus interface unit (BIU)
  - Execution unit (EU)
- 8086
  - Data bus size: 16 bit
  - Instruction queue: 6 bytes
- 20 bit address bus



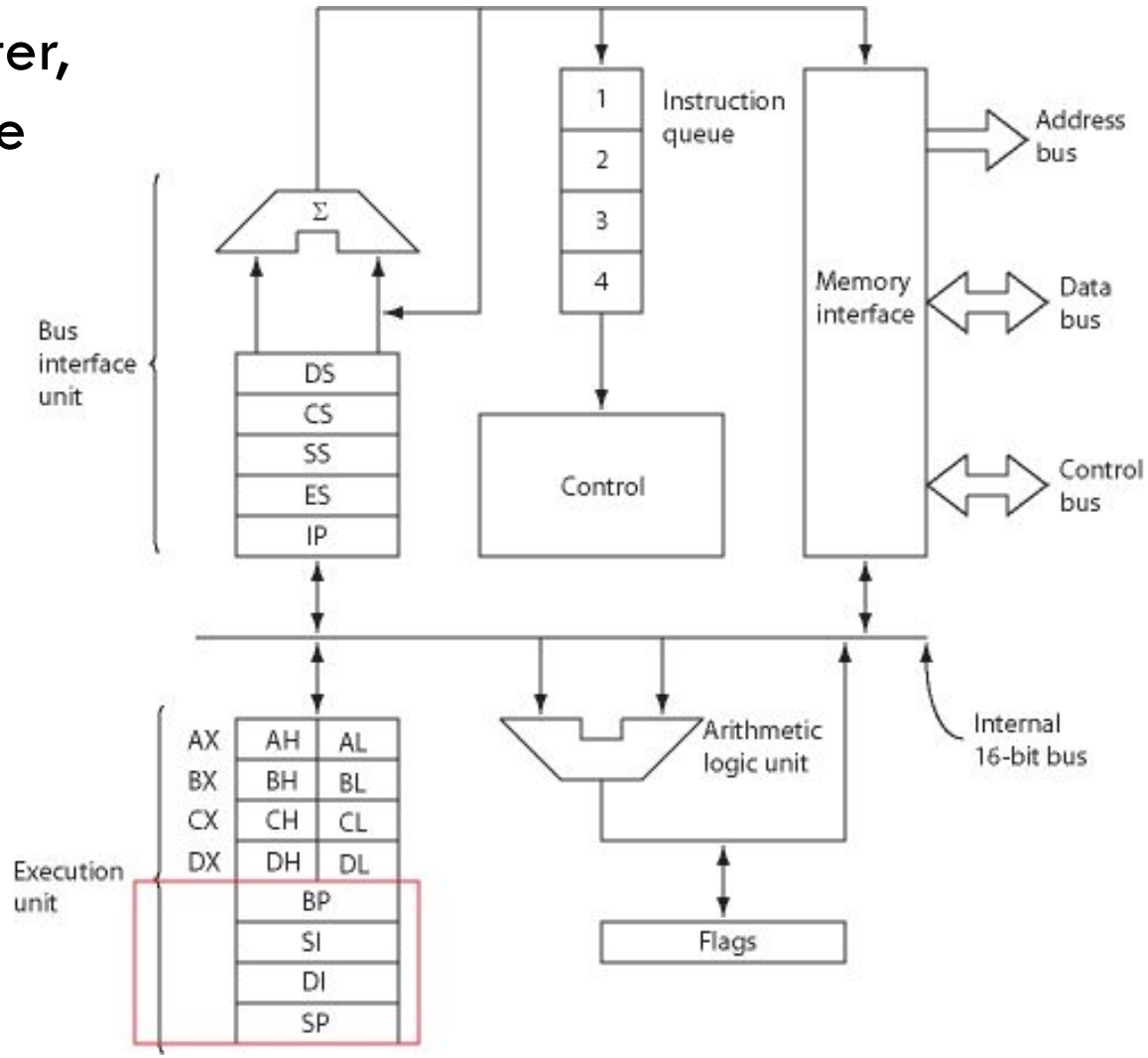
General Purpose Registers

- Used for arithmetic, logical, and other operations
- Can be configured as:
  - Four 16-bit registers
  - Eight 8-bit registers



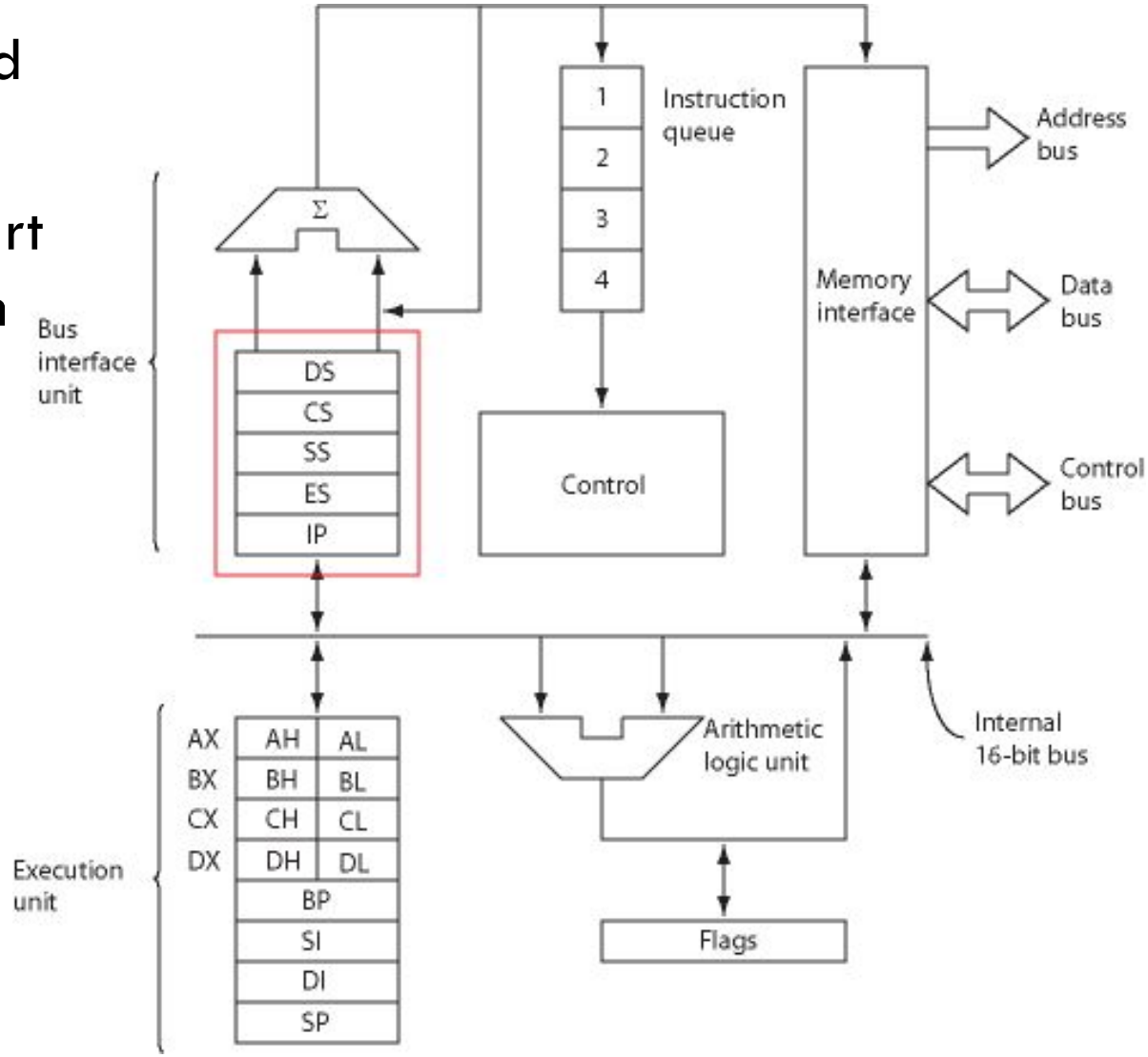
Index and Pointer Registers

- Base Pointer, Stack Pointer, Destination Index, Source Index
- Can only be used as 16 bit registers
- SI, DI used for addressing, and string op.
- BP, SP are used for maintaining the stack



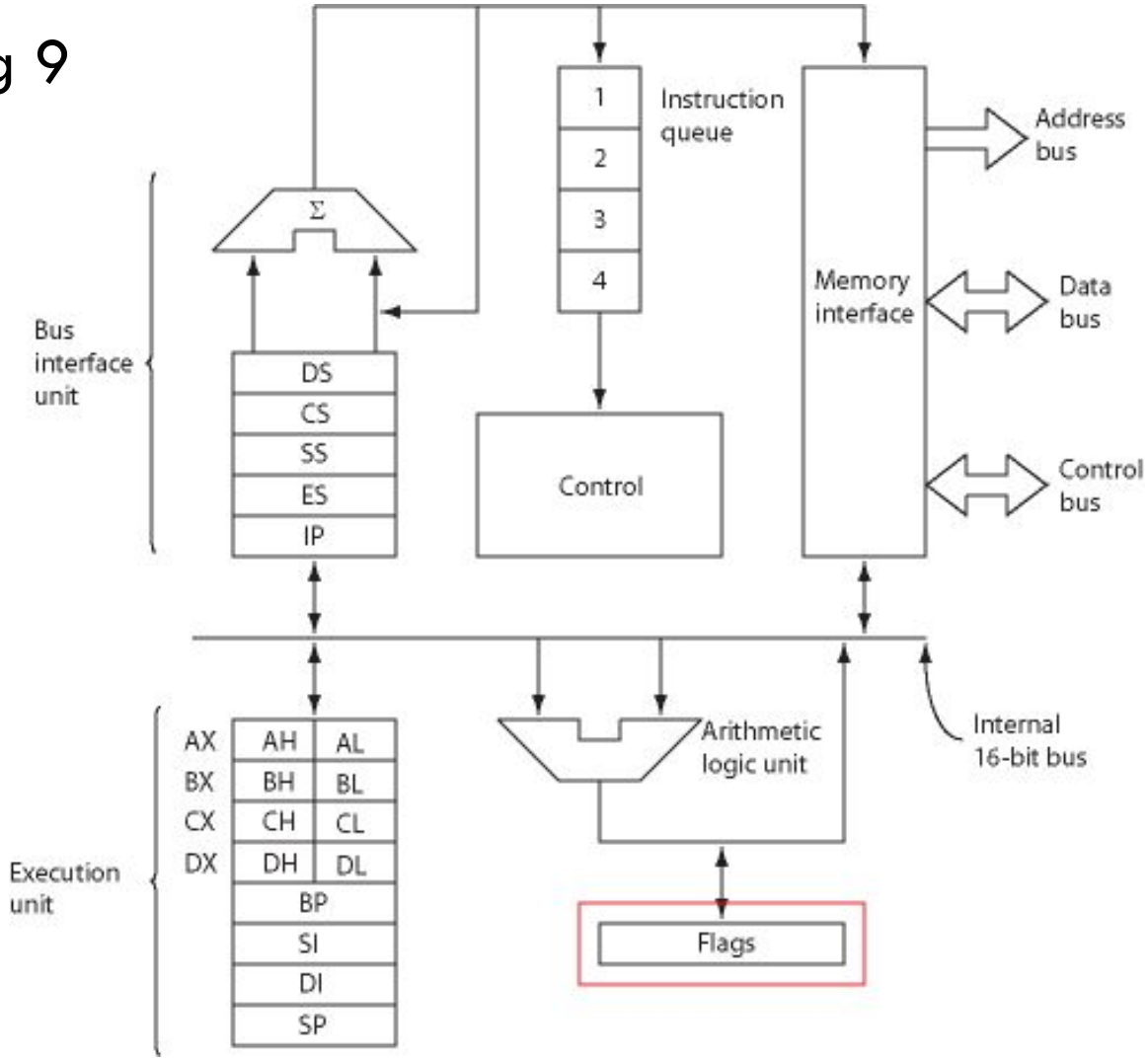
Segment Registers

- 16 bit registers labelled DS, CS, SS, ES
- Used to indicate the start of different segments in memory
- D – Data
- C – Code
- S – Stack
- E – Extra
- IP – Instruction Pointer



Flag Register

- 16-bit register containing 9 1-bit flags
- Gives status of the last instruction and the processor



Flag Register

- Overflow flag: Set if a signed overflow occurs
- Direction flag: Used to indicate direction of processing for string manipulations (0 = forward, 1 = backward)
- Interrupt enable flag: Setting this enables interrupts
- Trace (trap) flag: Used for debugging. When set processor executes only 1 instruction, then interrupts to call debugger
- Sign flag: Set to the MSB of the result of an operation
- Zero flag: Set if the result is zero
- Auxiliary carry flag: Set if there was a carry from or borrow to bits 0-3 in the AL register
- Parity flag: Set if the number of1's in result is even
- Carry flag: Set if there was a carry from or borrow to the MSB during last calculation

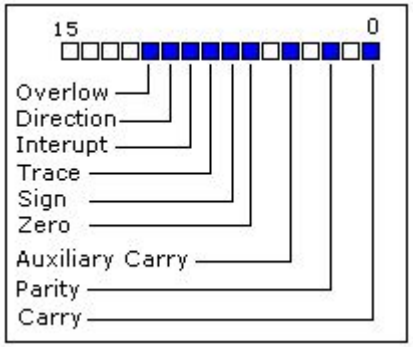


Image source: http://www.electronics.dit.ie/staff/tscarff/8086_registers/8086_registers.html

Programming Tools

DOSBox

- Required for those of you that have 64-bit machines and want to work on assignments at home
- Available at: http://www.dosbox.com/

Using DOSBOX

- Run DOSBOX
- Choose your working folder by 'mounting' your virtual C:\
  - Example: mount c "C:\Assembly"
  - Directory in quotations is where you will be storing your source code, masm.exe, link.exe, debug.exe
- Once the C:\ is mounted, type C: to switch to your directory in DOSBOX

DOS commands

- You will need to know simple DOS commands to navigate your folders in the command line

- **dir /p** – display contents of current directory (folders and files), one page at a time

- **cd FolderName** – change directory to the folder called 'FolderName' in your current directory
  - Alternatively you can type full directory path (C:\Windows\etc...)

DOS commands

- **cd ..** – Go up a level in the folder hierarchy
- **Prog.exe** – run program Prog.exe (must be in your current directory)
- **help** – displays a list of usable commands

Shortcuts

- Up arrow – previous command

- Tab – Scrolls through available files/directories that begin with whatever you have started typing
  - Use to avoid wasting time typing long directory pathways

Creating an Assembly file

- Use template.asm and save changes under different file name

  OR

- Create a .txt file
- Change file name extension to .asm
  - You may have to turn on the option to show file extensions on your computer
  - Method varies with OS – google if you can't find it

Programming Tools

- Course website has link to websites and tools you will need (these will also be included in As2 handout)
- After you have created an .asm file, you'll need to compile and link your program before you can run it
- This is done within DOSBOX from your mounted C:\

Programming Tools

- Compile, link, and run program
  - masm programName.asm
    - Include the file extension above
    - Hit enter 4 times
  - link programName
    - DO NOT include the file extension above
    - Hit enter 4 times
    - An exe file should have been generated in your directory
  - programName.exe
    - Runs the program
- To be demonstrated in class

Debugging

- Debug programName.exe
  - Type ? for command listing
- Commander that may be useful
  - Dump
    - Dumps a section of memory
  - go
    - Jumps to a line of code in the program
    - Is like running until a break point in a program
    - Can be used to skip over interrupt routines

Debugging

- trace
  - Allows you to step through the code one line at a time, allowing you to see the CPU state at any point
- unassemble
  - Displays the disassembly starting from the net instruction to be executed

There will be a section that goes more into depth on debugging later, just for now know

Notepad++

- Recommend notepad++ as assembly editor for this course
  - Highlights keywords
  - Good organization/formatting
  - Does not require an installer
- Available at: http://notepad-plus-plus.org/

Instruction Set

Assembly Language Statements

- Three types of statements
  - Directives or Pseudo-ops
  - Instructions
  - Macros

Assembly Language Statements

- Three types of statements
  - Directives or Pseudo-ops
    - Direct the assembler during the assembly process
    - Used for variable declaration and storage allocation
    - Non-executable and do not generate any machine language
    - e.g. segment, db, dw
  - Instructions
  - Macros

Assembly Language Statements

- Three types of statements
  - Directives or Pseudo-ops
  - Instructions
    - Instruct the processor to perform a task
    - Contains operation code (opcode)
    - Cause the assembler to generate machine language
    - e.g. mov, add
  - Macros

Assembly Language Statements

- Three types of statements
  - Directives or Pseudo-ops
  - Instructions
  - Macros
    - Permit the assembly language programmer to name a group of statements and refer to the group by the macro name
    - During the assembly process, each macro is replaced by the group of statements that it represents and assembled in place

Assembly Language Statements

- Take a look at template.asm (on the website for download)

- There are three memory segments visible here. In order of appearance:
  - Data – Directives and macros go here
  - Stack – More storage space (details on this later)
  - Code – Instructions go here

Directives: Data Allocation

[var-name] define-directive initial-value,[initial-value] …

- The define directive takes one of the five basic forms:
  - DB Define Byte ;allocates 1 byte
  - DW Define Word ;allocates 2 bytes
  - DD Define Doubleword ;allocates 4 bytes
  - DQ Define Quadword ;allocates 8 bytes
  - DT Define Ten Bytes ;allocates 10 bytes
- Examples:
  - var1 DB 'y' ;Use ASCII for number equivalent
  - var2 DB 79h ;(base 16)
  - var3 DB 11110010B ;(base 2)
  - var4 DB 62 ;(base 10)

Directives: Data Allocation

- Another example:
  - total DD 542803535
  - This would allocated four contiguous bytes of memory and initialize it to 542803535 (205A864Fh):

Address:	x	x+1	x+2	x+3
Contents:	4F	86	5A	20

Directives: Data Allocation

- Or:
  - total DB 20h,5Ah,86h,4Fh
  - This would allocated four contiguous bytes of memory and initialize it to 205A864Fh:

Address:	x	x+1	x+2	x+3
Contents:	20	5A	86	4F

Directives: Data Allocation

- Range of numeric operands
  - DB: -2^7 to $2^8 - 1$
  - DW: -2^{15} to $2^{16} - 1$
  - DD: -2^{31} to $2^{32} - 1$ or a short floating-point number (32 bits)
  - DQ: -2^{63} to $2^{64} - 1$or a long floating-point number
- Ranges depend on context (signed or unsigned)

Directives: Data Allocation

- Uninitialized Data and multiple initializations
  - Use DUP command
  - Number after DB indicates the number of bytes, and DUP(X) will fill the number of bytes with X
- Examples:
- test DB 10 DUP ('W') ;initializes 10 bytes to W
- test DB 1 DUP(?) ;Reserve 1 uninitialized byte
  - test DB DUP(?) is invalid – you must specify a number
- test DB 20 DUP(?) ;Reserve 20 uninitialized bytes

Directives: Data Allocation

- Multiple definitions:

```
sort     DB  'y'              ; ASCII of y = 79h
value    DW 25159             ; 25159 = 6247h
total    DD  542803535        ; 542803535 = 205A864Fh
```

- Memory allocation

Address: x	x+1	x+2	x+3	x+4	x+5	x+6
Contents: 79	47	62	4F	86	5A	20

- More examples:

```
message DB 'BYE',0Dh,0Ah   OR   message DB  'B'
                                DB  'Y'
                                DB  'E'
                                DB  0Dh
                                DB  0Ah
```

Directives: Defining Constants

- name EQU expression
  - Assigns the result of the expression to name.
- Example:

```
NUM_OF_STUDENTS EQU 90
                    …
MOV AX, NUM_OF_STUDENTS
```

Directives

- When allocating data with directives, space is always reserved in the **data segment**

8086 Instructions

- Instructions can be of various types
  - Data moving instructions
  - Arithmetic – add, subtract, increment, decrement, *etc.*
  - Control transfer – conditional, unconditional, call subroutine
  - Logic – AND, OR, XOR, shift/rotate and test
  - String manipulation – load, store, move, compare
  - Input/Output instructions
  - Other – setting/clearing flag bits, stack operations, *etc.*

MOV

mov destination, source

mov register, register

mov register, immediate

mov memory, immediate

mov register, memory

mov memory, register

Memory to memory forbidden.

- Examples

mov [response], bh

mov dx, [table1]

mov [name1+4], 'k'

Data Transfer Instructions

lea register, memory_address

- Loads effective address of variable
- Effective address – byte with lowest address of multi-byte object
  - (First char of a string)
- Examples

lea dx, mystring

Data Transfer Instructions

- XCHG exchanges 8, or 16-bit source and destination operands
- Examples
  - xchg ax, dx
  - xchg [response], cl
  - xchg [total], dx
- As in the mov instruction, both operands cannot be located in memory. Note that this restriction is applicable to most instructions

Arithmetic Instructions

- INC and DEC used to either increment or decrement the operands by one
  - inc destination
  - dec destination
- Examples
  - inc bx ; increment bx register by one
  - dec dl ; decrement 8-bit register

Arithmetic Instructions

- ADD used to add two 8 or 16-bit operands
- As with the mov instruction, add can also take the five basic forms depending on how the two operands are specified. The semantics of the add instruction are
- add destination, source => destination = destination + source

Arithmetic Instructions

- Examples:

Instruction	Before add		After add
	Source	Destination	Destination
add AX,DX	DX = AB62H	AX = 1052H	
add BL,CH	BL = 76H	CH = 27H	
add value,10H	—	value = F0H	
add DX,count	count = 3746H	DX = C8B9H	

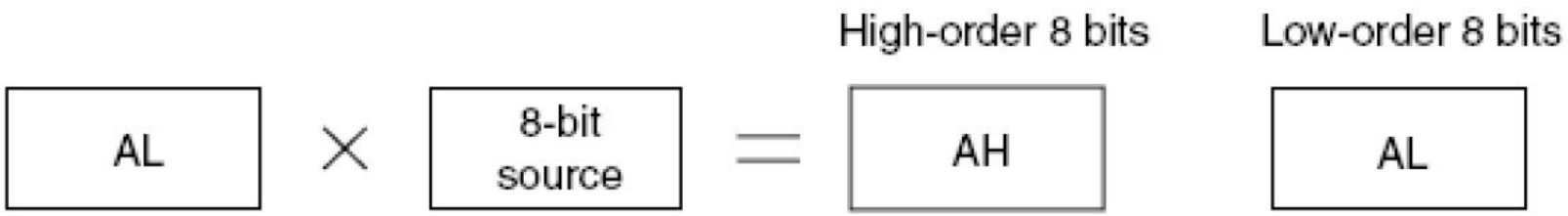
Arithmetic Instructions

- SUB used to subtract two 8, 16-bit operands
- sub destination, source => destination = destination - source
- Examples:

Instruction	Before sub		After sub
	Source	Destination	Destination
sub AX,DX	DX = AB62H	AX = 1052H	AX = 64F0H
sub BL,CH	CH = 27H	BL = 76H	BL = 4FH
sub value,10H	—	value = F0H	value = E0H
sub DX,count	count = 3746H	DX = C8B9H	DX = 9173H

Arithmetic Instructions

- MUL used to perform unsigned multiplication of a number with AL register
- mul source
- The source operand can be in a general-purpose register or in memory. Immediate operand specification is not allowed
- Example:
  - mov al, 10
  - mov dl, 25
  - mul dl



Arithmetic Instructions

- MUL is used to perform unsigned multiplication of a number with AL register
- Example:

```
mov al, 2
mov bl, 80h      ; 128
mul bl           ; Ax = 0100h (256)
```

Arithmetic Instructions

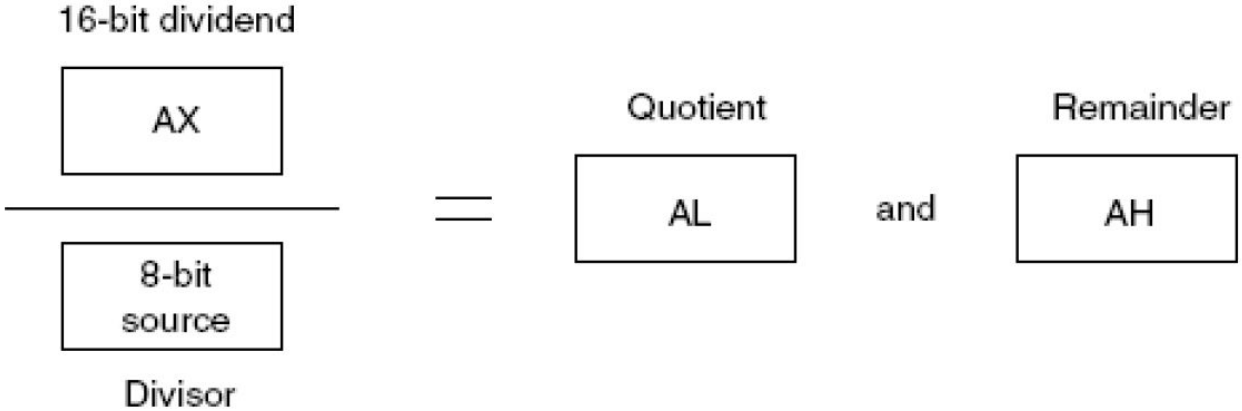
- IMUL used to perform signed multiplication of a number with AL register
- imul source
- The behaviour of the imul instruction is similar to that of mul
- The only difference to note is that the carry and overflow flags are set if the upper half of the result is not the sign extension of the lower half

```
mov al, 2
mov bl, f6h  ; -10
imul bl      ; ax = FFECh (-20)
```

- Note, here the carry and overflow flags are cleared, because AH contains the sign extension of the AL value

Arithmetic Instructions

- DIV, IDIV used to perform division of AX register
- div source (unsigned)
- idiv source (signed)
- Division generates two result components: a quotient and a remainder
- In multiplication, by using double-length registers, overflow never occurs. In division, divide overflow is a real possibility.
- The processor generates a special software interrupt when a divide overflow occurs



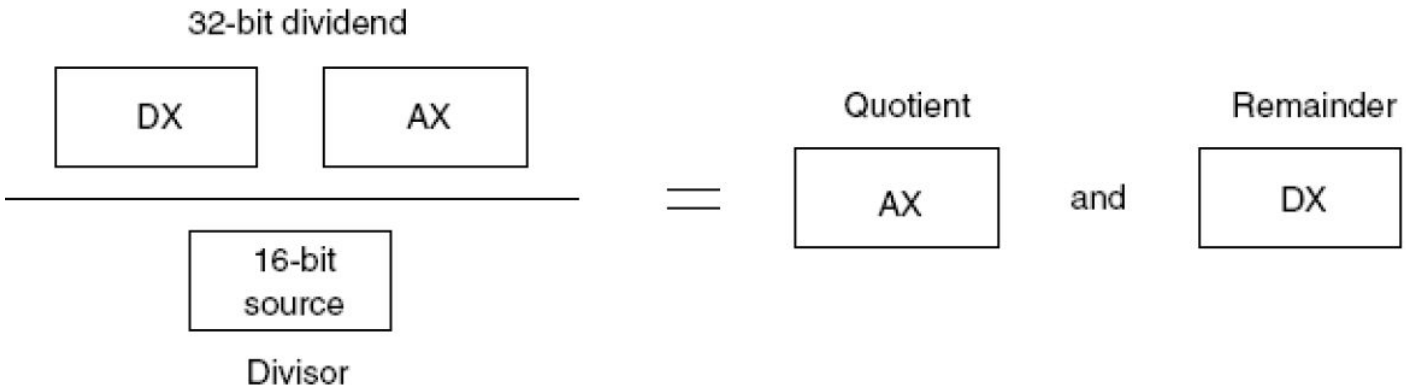
Arithmetic Instructions

- Example
  - mov ax, 251
  - mov cl, 12
  - div cl
- Leaves 20 (14h) in the al register and 11 (0bh) in the ah register
- Therefore, ax contains 0b14h after division

Arithmetic Instructions

Extension to larger operands:

- Both the multiplication and division instructions can be performed using 32-bit and 16-bit operands
  - Or, 64-bit and 32-bit operands on 32-bit system
- Example:



Arithmetic Instructions

- Need for sign extension
- Example: Perform signed division 8000h/400h
  - Here the divisor is larger than a byte, so dividend must be stored in DX AX
  - However, 8000h fits entirely within AX
  - So, AX needs to be sign extended to DX in order to maintain sign for division
- To aid sign extension instructions such as idiv, several instructions are provided:
  - cbw ; Convert byte to word
  - cwd ; Convert word to doubleword
  - cdq ; Convert doubleword to quadword